Unsupervised Learning

Krishna Sekhar
Padma Venkatraman

Recap and introductions

> **Note**
>
> Feel free to interrupt at any time to ask questions!

- **Last week** : Random forests, its advantages and pitfalls, and the importance of training on high quality data.
- **This week** : Forget about the labels, let the algorithm figure things out **unsupervised**

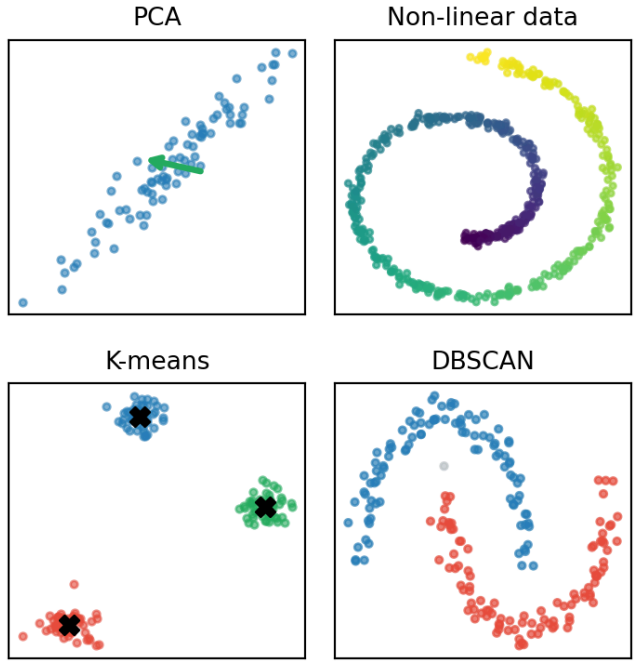
As we go through these lectures, I've tried to *highlight* jargon-y words, that might have a specific meaning in an AI/ML context

Unsupervised learning?

- **What** : Broadly, methods that find structure and patterns in data *without* pre-existing labels
- **Why** :
  - Sometimes labels don't exist, or are unreliable (garbage in, garbage out)
  - We want to discover structure in data that we didn't know to look for
  - Astronomical data can be high-dimensional (redshift, size, flux, SNR etc.). Unsupervised methods help us reduce complexity and identify which *features* matter.

This week

- Some math background (mostly linear algebra)
- Dimensionality reduction methods (PCA, UMAP)
- Clustering methods (K-Means, DBSCAN)
- Putting it all together on real data



Let’s get to the math

Representing dimensions

In math :

- Scalar : 0-dim ; e.g., height, weight, brightness, like $x = 1$
- Vector : 1-dim ; e.g., velocity, direction, Stokes parameters like $S = \begin{bmatrix} I & Q & U & V \end{bmatrix}$
- Matrix : 2-dim ; e.g., rotation matrix, covariance matrix like $R = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
- Tensor : n-dim ; e.g., stress-energy tensor, EM field tensors

Let’s get to the math

Representing dimensions

In code :

- Scalar : 0-dim, i.e., a variable `x = 1`
- Vector : 1-dim, i.e., an array `x = [1, 2, 3]`
- Matrix : 2-dim, an array/matrix

```
a =  [[1, 0],
      [0, -1]]
```

- Tensor : n-dim, a multi-dimensional array

```
t = [[[1, 2], [3, 4]],
     [[5, 6], [7, 8]]]
```

So if we want to do some matrix multiplication :

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

```
1 import numpy as np
2 a = np.asarray([[1, 0], [0, -1]])
3 b = np.asarray([[1], [1]])
4
5 c = a @ b
6 print(c)
```

Notice that for this to work, the dimensions had to work out. We multiplied a 2x2 matrix with a 2x1 column vector.

If we reverse the operation, it will error out.

```
try:
    c = b @ a
except ValueError as e:
    print(f"ValueError: {e}")
```

Some more math

Variance and Co-variance

- The variance measures how “spread out” data is around the mean - *i.e.*, how *noisy*
- The variance is also the square of the standard deviation

$$\mathrm{Var}(X) = \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

where μ is the mean of data, and x_i is an individual data point.

Variance

$$\mathrm{Var}(X) = \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

- The square also squares values with units, so say variance of something in m becomes m^2
- So it's useful to consider the standard deviation, preserves units, easier for direct comparisons

$$\sigma = \sqrt{\mathrm{Var}(X)} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

Covariance

Measures how two variables change with each other

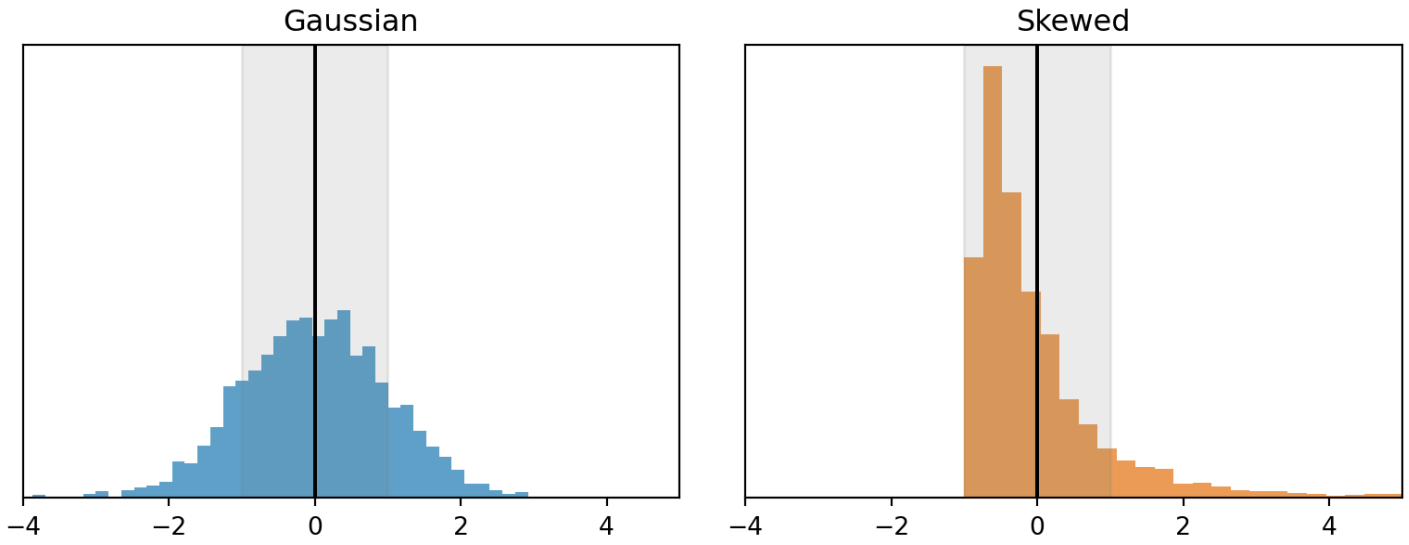
$$\mathrm{Cov}(X, Y) = \frac{1}{N} \sum (x_i - \mu_X)(y_i - \mu_Y)$$

The variance then becomes a special case, the covariance of a variable with **itself**. The problem of units becomes even worse, because X and Y can have **different** units. Define a correlation coefficient

$$r = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

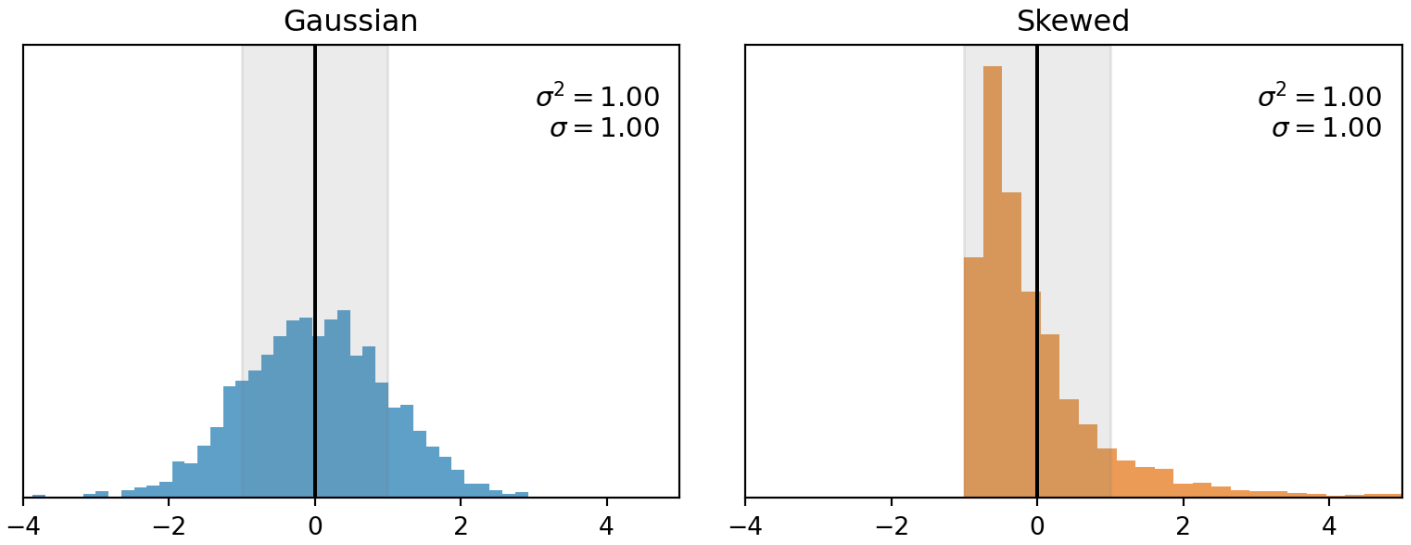
Variance isn’t the whole story

Think about it: which distribution has the **higher variance** — the Gaussian or the skewed one?



Variance isn’t the whole story

They have the **same variance** — yet they describe very different data.



Same σ, different skew — **PCA ranks directions by variance alone**.

Some more math

Eigenvalues and eigenvectors

- In general, a matrix is a **transformation machine** that stretches, rotates, and shears the vector it is acting on.
- An eigen*vector* is a special vector that only gets scaled, no rotation or shear. The eigen*value* is how *much* it gets scaled.

$$A\vec{v} = \lambda\vec{v}$$

eigen /ˈaɪgən/ *adjective · German*

proper, inherent, or characteristic

A is an $N \times N$ matrix, λ is the eigen*value*, $\vec{v}$ is the eigen*vector*

Eigenvectors in the wild

Discuss

Where have you run into eigenvectors before?

Eigenvectors and Eigenvalues

```
import numpy as np

A = np.array([[2, 1], [1, 2]], dtype=float)
print(A)
```

```
# eigh is for symmetric matrices — guarantees real eigenvalues
eigval, eigvec = np.linalg.eigh(A)
print(f"Eigenvalues are {eigval}")
print(f"Eigenvectors are\n{eigvec}")
```

```
# Eigenvectors are the COLUMNS of eigvec, not the rows
print(f"Eigenvalue {eigval[0]} is for eigenvector {eigvec[:, 0]}")
print(f"Eigenvalue {eigval[1]} is for eigenvector {eigvec[:, 1]}")
```

Distance metrics and optimization

Distance metrics

How do you calculate the distance from one point to another?

Distance Metrics

We'll cover a handful of common distance metrics that are commonly applied to AI/ML methods. This is **not** a complete list!

- L1 distance
- L2 distance
- Cosine similarity/Cosine distance

Honorable mentions:

- Mahalanobis distance
- Minkowski distance

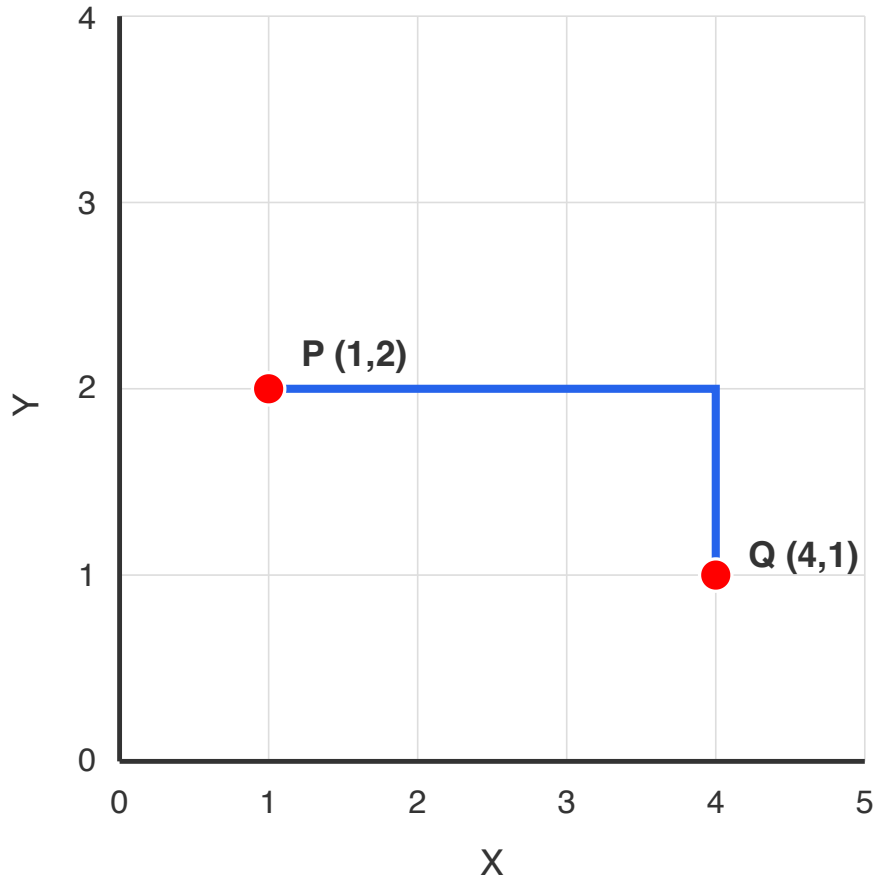
L1 distance

Also called : Manhattan distance, taxicab distance

Measures the distance by summing the **absolute difference** in co-ordinates between two points.

In two dimensions :

$$d = |x_1 - x_2| + |y_1 - y_2|$$

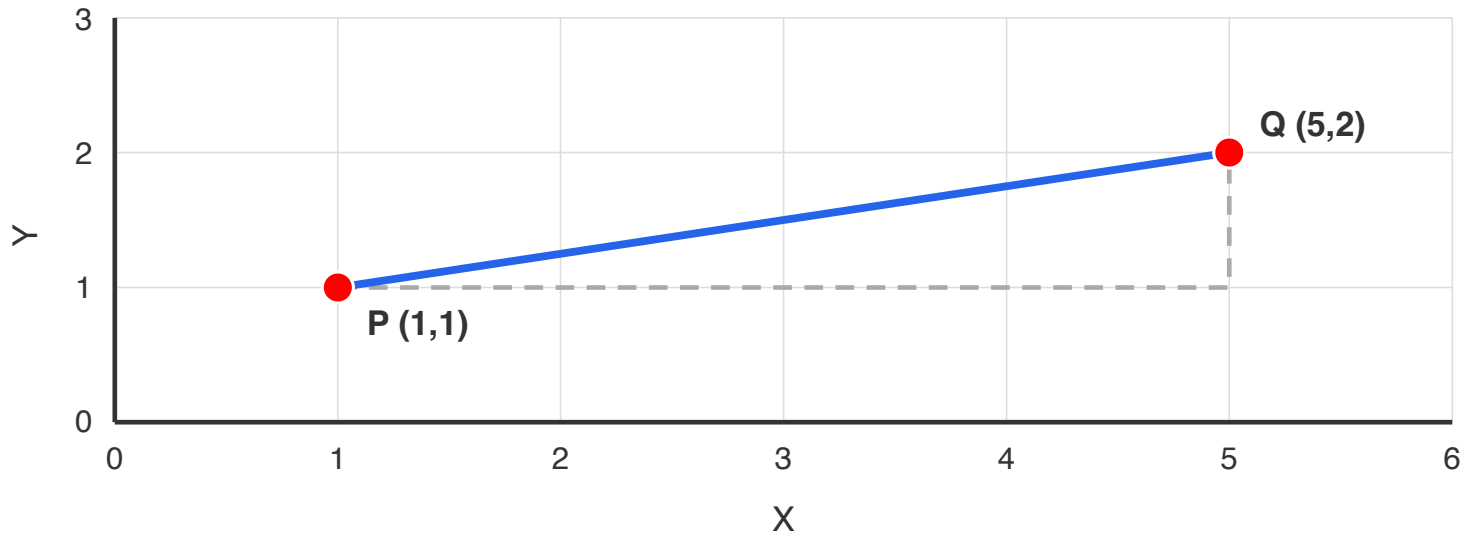


L2 Distance

a.k.a Euclidean distance

The “straight line distance” between two points. Can also think of this as the hypotenuse in 2D

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$



Cosine Distance

The cosine distance is

$$d_c = 1 - S_c = 1 - \cos\theta$$

$$d_c = \begin{cases} 0, & \rightarrow\rightarrow \\ 1, & \rightarrow\uparrow \\ 2, & \rightarrow\leftarrow \end{cases}$$

This gives you a positive increasing value that corresponds to the orientation of two vectors.

Distance Metrics Recap

- L1 (Manhattan/taxicab) $d = |x_1 - x_2| + |y_1 - y_2|$
- L2 (Euclidean) $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- Cosine distance $d = 1 - S_c = 1 - \frac{\vec{A} \cdot \vec{B}}{\|A\|\|B\|}$

Other metrics :

- Mahalanobis distance : Distance from a point to a probability distribution
- Minkowski distance : Generalized version of L1 and L2; Valid for any “n” (Ln)

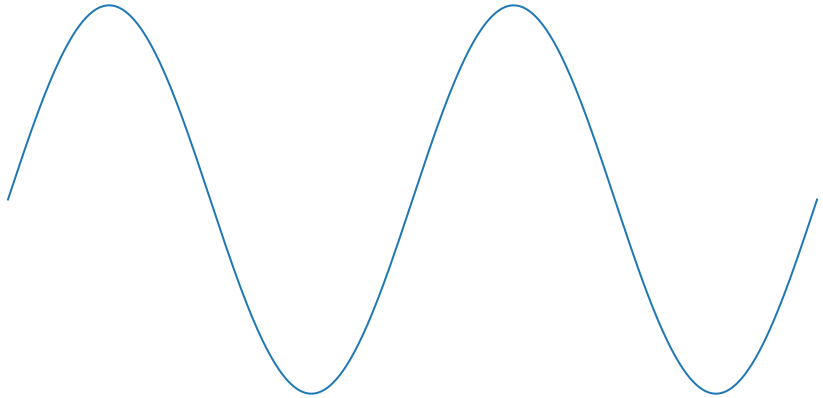
Which metric, when?

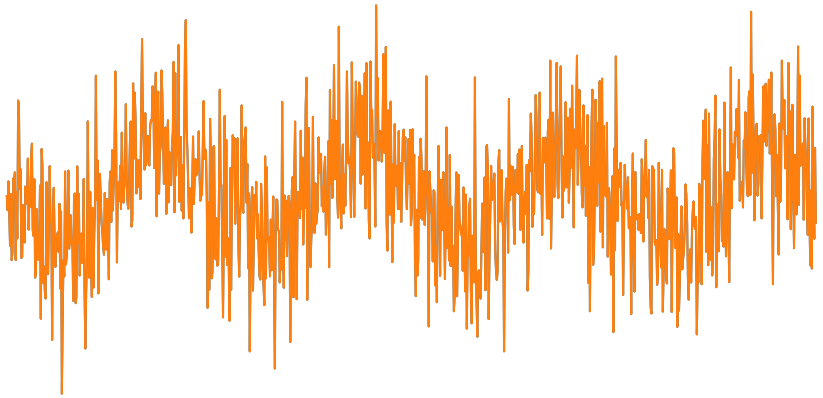
Discuss

You’re comparing two galaxy spectra at different brightness. L1, L2, or cosine — which would you reach for, and why?

Optimization

Given a model and some data, how do we tune the model parameters to best fit the data?





Loss functions

How do we measure how wrong a model is, given the data? i.e., how **far away** is our model from the data?

Ideally we want a single number to quantify this, which we want to make as small as possible. This is called the *loss*.

We can re-purpose our distance metrics to calculate loss

L2 loss : $\square = \frac{1}{n}\sum(y_i - \hat{y_i})^2$

L1 loss : $\square = \frac{1}{n}\sum|y_i - \hat{y_i}|$

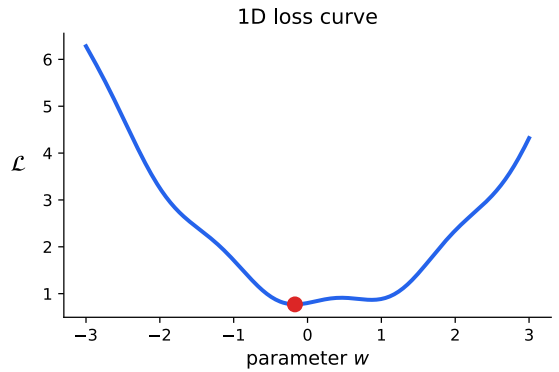
where y_i is the data and $\hat{y_i}$ is the model prediction.

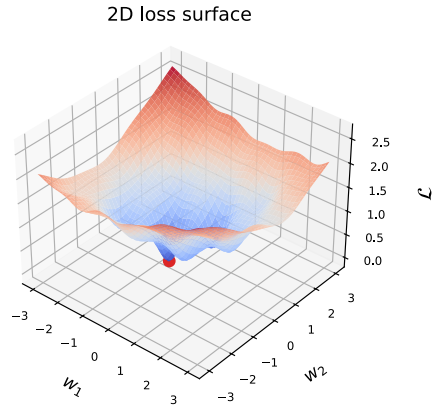
Loss must always be ≥ 0

Loss surface

The loss ☐ depends on the model parameters.

If you have one parameter, loss traces out a **curve**. Two (or more) parameters → a **surface**.





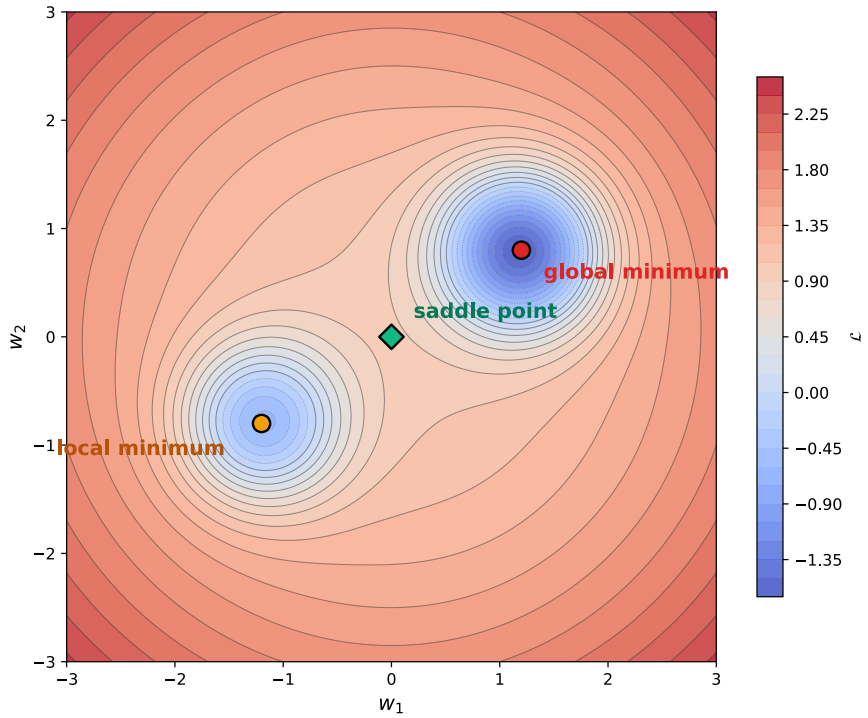
The goal of optimization: find the parameter values where ☐ is smallest.

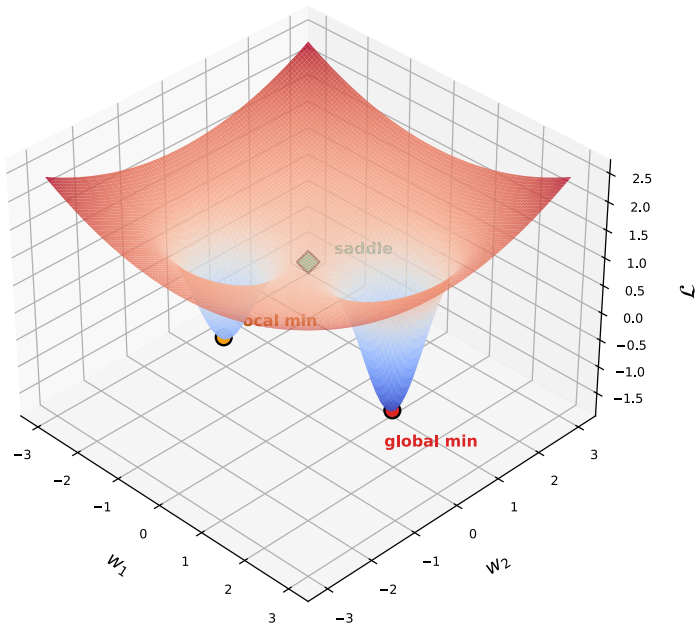
Loss surface

Not all low points are equal:

- **Global minimum** — the lowest point on the **entire** surface. This is what we want.
- **Local minimum** — a low point, but not the lowest. Optimizers can often get stuck here.
- **Saddle point** — flat in some directions, but not a minimum. Misleading in high dimensions.

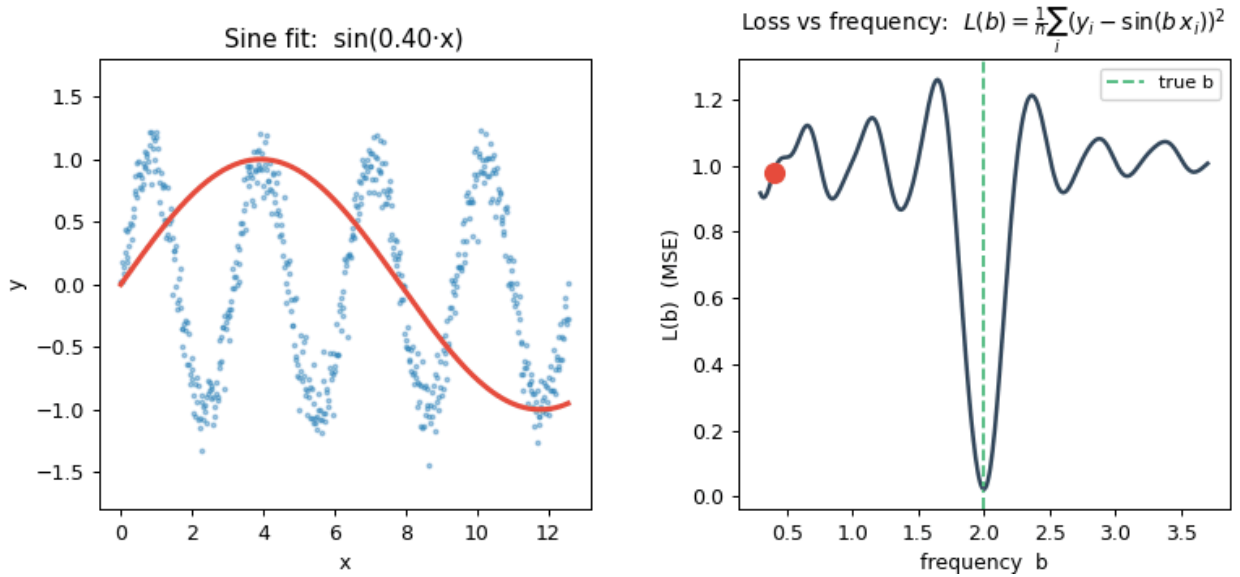
Loss surface





A loss curve in the wild

Fit `sin(b·x)` to noisy data and sweep the frequency `b`. The loss traces out a **bumpy, non-convex** curve — one deep global minimum, plus shallower local dips.



Can we do better than sweeping all the parameters to figure out the best fit?

Gradient descent

How do we find the minimum? We can't just try every point — the parameter space is too large.

The **gradient** $\nabla \square$ points in the direction of steepest *increase*.

So we step in the **opposite direction**:

$$w_{t+1} = w_t - \eta \, \nabla \square(w_t)$$

where η is the *learning rate* — how big a step we take each iteration.

Convergence

We iterate gradient descent until a **stopping criterion** is met:

- $\Delta\square < \epsilon$ — loss stopped decreasing between iterations
- $\|\nabla\square\| \approx 0$ — gradient is near zero, we're at a flat region
- Maximum iterations reached — a practical safety bound

Convergence

Discuss

When do we stop (i.e., determine something to be converged?)